C'est un superbe schéma d'architecture ! On y voit très clairement le flux cible : l'événement brut (JSON) arrive dans le **Bucket IN_API_EVENT**, Spark le récupère en streaming continu ("Get Event"), le transforme ("Process") et l'écrit proprement dans la table **Iceberg** ("Write to Refined data storage") pour que Starburst puisse le lire en SQL. Le tout avec un objectif de moins d'une minute de latence.

Pour réaliser cela, ton code Python va devoir évoluer. Il ne va plus écrire de simples fichiers Parquet, mais utiliser le connecteur **Apache Iceberg** pour Spark.

Voici le code source de ton application Spark Streaming (streaming_app_iceberg.py) qui correspond exactement à cette architecture et aux spécifications qu'on a vues ensemble.

Le Code PySpark (Streaming vers Iceberg)

Python

```
from pyspark.sql import SparkSession

from pyspark.sql.types import StructType, StructField, StringType, TimestampType

from pyspark.sql.functions import current_timestamp


# 1. Initialisation de la Session avec la configuration Iceberg

# Note: Dans un vrai cluster d'entreprise, les lignes de configuration du catalog

# sont souvent déjà définies par l'IT globale, mais on les force ici pour le PoC.

spark = SparkSession.builder \

    .appName("poc-streaming-iceberg") \

    .config("spark.eventLog.enabled", "false") \

    .config("spark.sql.extensions",
"org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions") \

    .config("spark.sql.catalog.dh_poc_ice", "org.apache.iceberg.spark.SparkCatalog") \

    .config("spark.sql.catalog.dh_poc_ice.type", "hadoop") \

    .config("spark.sql.catalog.dh_poc_ice.warehouse", "s3a://bu002i004226/") \

    .getOrCreate()


spark.sparkContext.setLogLevel("WARN")
```

```
print("### DÉMARRAGE DU STREAMING VERS ICEBERG ###")

# 2. Le Schéma Statique (Pour éviter le inferSchema très lent)
# On correspond exactement à la spécification CloudEvent
event_schema = StructType([
    StructField("specversion", StringType(), True),
    StructField("type", StringType(), True),
    StructField("source", StringType(), True),
    StructField("subject", StringType(), True),
    StructField("id", StringType(), True),
    StructField("time", TimestampType(), True),
    StructField("datacontenttype", StringType(), True),
    StructField("data", StringType(), True) # On garde le payload JSON brut en String
])

# 3. GET EVENT : Lecture du flux JSON depuis le bucket RAW
# Remplace par le vrai chemin de ton dossier RAW (Bucket IN_API_EVENT)
df_raw = spark.readStream \
    .format("json") \
    .schema(event_schema) \
    .load("s3a://bu002i004226/poc_streaming/input_raw/")

# 4. PROCESS : Transformation des données
# On ajoute la fameuse colonne technique demandée pour la traçabilité
df_processed = df_raw.withColumn("dh_poc_insert_timestamp", current_timestamp())

# 5. WRITE TO REFINED DATA STORAGE : Écriture dans la table Iceberg
# Le Trigger à 30 secondes permet de respecter ton SLA "Target: En moins d'une minute"
```

tout en regroupant un peu les données pour ne pas faire exploser Starburst avec de trop petits fichiers.

```
query = df_processed.writeStream \
  .format("iceberg") \
  .outputMode("append") \
  .trigger(processingTime="30 seconds") \
  .option("checkpointLocation", "s3a://bu002i004226/poc_streaming/checkpoint_iceberg/") \
  .toTable("dh_poc_ice.pocspark") # Nom du catalogue + nom de la table


query.awaitTermination()
```

⚠️ **Ce qu'il faut ajouter dans ton script Bash (submit_streaming.sh)**

Pour que Spark comprenne ce code, il doit télécharger les "moteurs" (les fichiers .jar) d'Iceberg au moment de démarrer. Dans ton script bash actuel, tu dois rajouter l'option --packages (si tu ne l'as pas déjà pour AWS) ou y ajouter Iceberg.

Ajoute cette ligne dans ton spark-submit (vérifie bien la version de ton Spark, ici j'ai mis la version pour Spark 3.3 et Scala 2.12, c'est le standard actuel, mais à adapter selon ton cluster) :

Bash

```
 --packages org.apache.iceberg:iceberg-spark-runtime-3.3_2.12:1.3.1 \
```

Pourquoi ce code est "Prod-Ready" pour ton schéma :

1. **Zéro "inferSchema"** : En lui donnant event_schema dès le départ, Spark ne perd pas de temps à lire les fichiers en double. Il prend le JSON, le valide contre le schéma, et passe à la suite.
2. **La colonne technique** : Le withColumn("dh_poc_insert_timestamp", current_timestamp()) marque exactement l'heure d'arrivée dans ton système, ce qui comble la différence entre le time de l'événement source et la réalité de l'intégration.
3. **Le Trigger de 30 secondes** : Plutôt que de tourner à vide en permanence, Spark fait une ronde toutes les 30 secondes, rassemble les JSON arrivés entre temps, et fait un seul "Commit" propre dans Iceberg. Ton SLA de 1 minute sur le schéma est largement respecté !

Avec ça, tu as la boucle complète. Starburst n'aura plus qu'à interroger la table dh_poc_ice.pocspark de son côté ! Prêt à tester ça à ton retour de télétravail ?